

Enterprise SQL Database

Enterprise Database Workspace & Runtime Schema Management Platform.

Author: Keeno Smith

Project Type: Full-Stack Database Engineering Platform / Enterprise Data Management Workspace

Primary Domain: Database Engineering, SQL, PostgreSQL, Relational Data Management.

Role: Database Application Engineer / Full-Stack Software Engineer

Primary Technology Stack: PostgreSQL, Prisma ORM, Node.js, Express, React, Vite

Core Database Technologies: PostgreSQL SQL, Runtime DDL, information_schema, PostgreSQL constraint metadata, relational modelling, foreign keys, primary keys, unique constraints, nullable constraint, defaults, identity columns, schema introspection

Supporting Technologies: React 19, React Router, native Fetch API, JavaScript ES Modules, dnd-kit, react-rnd, OXlint, Prisma CLI

Executive Summary

A full-stack database engineering platform designed around **PostgreSQL database management, relational data modelling, runtime schema evolution and SQL-driven workspace operations**.

The platform provides an interactive environment for managing structured relational data while exposing database concepts that are normally handled through dedicated database administration tooling. Users can create and manage tables, define and modify columns, establish primary and foreign-key relationships, configure constraints and defaults, create and manipulate records, inspect the physical database schema, execute SQL queries and work with relational structures through a browser-based workspace.

The defining engineering capability is the platform's ability to **translate application-level schema definitions into PostgreSQL DDL operations at runtime**. Rather than operating against a fixed collection of application models, the system can create physical PostgreSQL tables, alter their columns and constraints, introspect the resulting database structure and maintain corresponding application metadata.

This makes the project primarily a **PostgreSQL and database systems engineering implementation**, with the React and Express layers providing the application interface through which database operations are orchestrated.

Primary Objective

The primary objective was to engineer a browser-based database workspace capable of providing controlled management of PostgreSQL data structures and records through an application layer.

The platform addresses several database-management requirements within one system:

- Relational table management
- Runtime table creation and deletion
- Runtime column creation, modification and deletion
- SQL DDL generation
- Database schema introspection
- Primary-key management
- Foreign-key relationships
- Unique constraints
- Nullable/non-nullable fields
- Default values
- Auto-incrementing identity columns
- Runtime record CRUD
- SQL query execution
- Data searching, sorting and filtering
- Database relationship exploration
- Schema-driven record forms
- Database health monitoring
- Synchronisation between database structures and application metadata

What The Application Does

The application provides a **database workspace for interacting with PostgreSQL relational data structures and records**.

Users can work with core business tables such as Products, Categories and Suppliers while also creating custom tables whose structures are defined dynamically.

The custom-table system supports a hierarchy of:

Workspace Table → Columns → Constraints → Relationships → Records

The frontend dynamically interprets the resulting schema to determine how records should be displayed and edited.

The application therefore operates across both major database concerns:

Data Definition

Creating and modifying tables, columns, types and constraints.

Data Manipulation

Creating, reading, updating and deleting records within those structures.

It additionally provides a query subsystem for executing database queries and dedicated views for schema, relationships, query history and query construction.

Database Architecture

The database architecture is built around **PostgreSQL as the authoritative physical relational database**, with Prisma providing structured application-level database access and metadata persistence.

The architecture uses a hybrid approach:

React

→ database workspace interface

Node.js / Express

→ REST API and application orchestration

Prisma ORM

→ application/database metadata models and structured database access

PostgreSQL

→ physical relational tables, records, constraints and runtime schema

SQL / PostgreSQL DDL

→ runtime database structure manipulation

This separation allows the application to use an ORM for conventional application persistence while retaining direct SQL capabilities where ORM abstractions are insufficient for dynamic database schema manipulation.

PostgreSQL Engineering

PostgreSQL is the central technical component of the project.

The backend interacts with PostgreSQL at both the application-data level and the database-definition level.

The platform works with PostgreSQL concepts including:

- Tables
- Columns
- Data types
- Primary keys
- Foreign keys
- Unique constraints
- Nullability
- Default values
- Identity/auto-increment fields
- Foreign-key references
- Constraint metadata
- Physical record counts
- Database schema metadata

The implementation also uses PostgreSQL-specific facilities including:

- `information_schema`
- `pg_constraint`
- PostgreSQL constraint metadata
- PostgreSQL error codes
- `ALTER TABLE`
- `CREATE TABLE`
- `DROP COLUMN`
- `RENAME COLUMN`
- `ALTER COLUMN TYPE`
- `SET/DROP NOT NULL`
- `SET/DROP DEFAULT`
- `ADD CONSTRAINT`
- `DROP CONSTRAINT`
- PostgreSQL identity columns

This provides direct exposure to database engineering beyond conventional application CRUD.

Runtime Database Schema Management

One of the project's strongest technical capabilities is **runtime schema evolution**.

The platform can dynamically create PostgreSQL tables based on user-defined schema specifications.

A table definition can include:

- Column name
- Data type
- Primary-key status
- Auto-increment status

- Nullability
- Uniqueness
- Default value
- Foreign-key status
- Referenced table
- Referenced column

These definitions are validated before being converted into PostgreSQL SQL.

The system can subsequently modify the physical database structure through operations including:

Table

- Create
- Rename
- Delete

Column

- Add
- Rename
- Change data type
- Change nullability
- Change default
- Add uniqueness
- Remove constraints
- Delete

This represents a genuine **runtime schema-management layer**, rather than simply allowing users to create records within predefined tables.

SQL Generation Architecture

A dedicated SQL-builder layer encapsulates PostgreSQL DDL generation.

The implementation provides SQL builders for:

- `CREATE TABLE`
- `ALTER TABLE ... ADD COLUMN`
- Foreign-key creation
- Column renaming
- Data-type alteration
- Nullability changes
- Default changes
- Unique constraints
- Constraint removal
- Column removal

The SQL generation layer separates SQL construction from higher-level controller logic.

For example, application-level definitions are transformed into PostgreSQL column definitions containing combinations of:

`INTEGER`, `DECIMAL`, `VARCHAR`, `TEXT`, `BOOLEAN`, `DATE`, `TIMESTAMP`

alongside:

- `NOT NULL`
- `UNIQUE`
- `DEFAULT`
- `PRIMARY KEY`
- PostgreSQL identity generation

This creates a dedicated abstraction between the application's schema model and PostgreSQL's physical DDL representation.

SQL Identity Safety

Dynamic database systems introduce an important distinction between **SQL values** and **SQL identifiers**.

The implementation explicitly handles dynamically generated table and column names through an identifier-quoting function.

Identifiers are:

- Required to be valid strings.
- Trimmed.
- Length validated.
- Escaped when containing double quotes.
- Wrapped as PostgreSQL quoted identifiers.

This is particularly important because the application dynamically generates SQL against tables and columns that can be created or renamed through the workspace.

The architecture therefore does not treat dynamically supplied schema names as ordinary string fragments.

Database Type Normalisation

The system maintains an application-level type model while also interpreting PostgreSQL-native type representations.

Supported application types include:

- `INTEGER`
- `DECIMAL`
- `VARCHAR`
- `TEXT`
- `BOOLEAN`
- `DATE`
- `TIMESTAMP`

PostgreSQL representations such as:

- `int2`
- `int4`
- `serial`
- `int8`
- `bigserial`

- `varchar`
- `bpchar`
- `numeric`
- `decimal`
- `bool`
- `timestamp`
- `timestampz`

are normalised into the application's schema representation.

This allows PostgreSQL's physical schema to be translated into a consistent application-level schema model for rendering and management.

Relational Database Modelling

The platform implements a relational data model around Products, Categories and Suppliers.

The core relationship is:

Category

→ Products

Supplier

→ Products

Products contain references to their associated category and supplier through foreign-key identifiers.

The database workspace extends this concept by allowing custom tables to define their own foreign-key relationships.

Foreign keys require:

- Referenced table
- Referenced column
- Valid identifier definitions
- Referenced-object existence
- Compatible data types

The backend validates referenced structures before applying relationship changes to PostgreSQL.

Constraint Engineering

Database integrity is enforced through multiple layers of schema validation.

The system supports:

Primary Keys

- Primary-key identification
- Single-primary-key validation
- Identity/auto-increment integration
- Protection against unsupported primary-key alterations

Foreign Keys

- Referenced table validation
- Referenced column validation
- Referenced type compatibility
- Foreign-key metadata resolution
- Runtime constraint creation

Unique Constraints

- Column-level uniqueness
- Runtime unique constraint creation
- Constraint removal

Nullability

- Nullable fields
- Required fields
- Runtime `SET NOT NULL`
- Runtime `DROP NOT NULL`

Before changing an existing nullable column into a required column, the system checks for existing `NULL` values and rejects the operation when existing data would violate the new constraint.

Default Values

The implementation validates defaults according to the underlying data type.

Examples include:

- Boolean values
- Numeric values
- Timestamp defaults
- Escaped string defaults
- `CURRENT_TIMESTAMP`

These constraints demonstrate database-integrity considerations rather than relying exclusively on frontend validation.

Database Schema Introspection

The platform includes a dedicated schema-introspection subsystem.

PostgreSQL metadata is queried through database catalog and information-schema facilities to reconstruct the physical database structure.

The resulting schema representation includes information such as:

- Table names
- Column names
- Data types
- Column positions
- Primary-key status
- Foreign-key status
- Nullable status
- Unique status

- Default values
- Auto-increment status
- Foreign-key references
- Physical record counts

Internal Prisma/workspace metadata tables are excluded from the user-facing schema representation.

This allows the application to derive its database view from the **actual physical PostgreSQL schema**, rather than assuming that application metadata always represents the current database state.

Metadata Synchronization

The platform maintains application-level metadata for dynamically managed workspace tables and columns while PostgreSQL remains responsible for the physical database structure.

This creates a synchronisation requirement between:

Application Metadata

WorkspaceTable
→ WorkspaceColumn

and:

Physical PostgreSQL Schema

Table
→ Column
→ Constraint
→ Record

The backend performs physical-schema validation and metadata updates as part of schema-management operations.

It also detects stale workspace metadata when an expected physical PostgreSQL table no longer exists and removes the invalid metadata representation.

This provides a mechanism for maintaining consistency between the application's database workspace model and the underlying relational database.

Transactional Schema Operations

Schema operations are implemented with transactional database workflows where physical PostgreSQL changes and associated metadata changes need to remain coordinated.

For example, adding a workspace column involves:

Request

- validation
- physical table verification
- PostgreSQL `ALTER TABLE`
- foreign-key constraint creation where required
- metadata synchronisation

The backend also re-validates relevant physical structures inside transactional operations rather than relying exclusively on state captured before the transaction begins.

This approach reduces the risk of application metadata representing a schema state that was never successfully applied to PostgreSQL.

Dynamic Record Management

The database workspace is not limited to DDL.

It also provides dynamic CRUD operations against runtime-created tables.

Supported operations include:

- Retrieve records
- Create records
- Update records
- Delete records

For custom tables, record structures are derived from the table's current column definitions.

The frontend uses those definitions to generate appropriate data-entry controls for:

- Numeric values
- Dates
- Timestamps
- Boolean values
- Text values

Primary-key and auto-increment behaviour is accounted for when creating records.

This creates a **schema-driven CRUD layer**, where the database schema determines the application's data-management interface.

Database Query Subsystem

The platform exposes a dedicated SQL query API through:

`POST /api/query`

with the query supplied as the request payload.

This separates direct query execution from conventional resource CRUD operations.

The application also contains dedicated Query and Query Builder functionality, supported by the broader query-history and database-workspace architecture.

This provides two complementary database interaction models:

Structured database operations

Tables, columns, relationships and records.

SQL-driven database operations

Direct query execution through the query subsystem.

Database API Architecture

The backend follows a modular Express routing architecture.

Core database endpoints include:

Products

`/api/products`

Categories

`/api/categories`

Suppliers

`/api/suppliers`

Workspace Tables

`/api/tables`

Query Execution

`/api/query`

The workspace table API follows a hierarchical REST structure:

`/api/tables`

→ tables

`/api/tables/:id/columns`

→ columns/schema

`/api/tables/:id/records`

→ records/data

This mirrors the underlying relational structure within the API design.

ORM and Persistence Strategy

Prisma ORM provides structured database access for the application layer.

The project also uses direct PostgreSQL SQL where dynamic schema manipulation requires capabilities beyond ordinary ORM model operations.

This hybrid strategy is technically significant:

Prisma

→ structured application/database interaction

Raw PostgreSQL SQL

→ dynamic physical schema manipulation

This avoids forcing runtime-created database structures into a static ORM model while still benefiting from Prisma's application-level database abstraction.

Backend Engineering

The Node.js/Express backend provides:

- REST API routing
- Database orchestration
- Schema validation
- SQL generation
- PostgreSQL metadata access
- Runtime DDL execution
- Record CRUD
- Query execution
- Database health checks
- Error handling
- Environment configuration

The application separates:

Routes

→ endpoint definitions

Controllers

→ application/database operations

SQL Builders

→ PostgreSQL statement generation

Prisma

→ database access and metadata persistence

This produces a modular backend architecture rather than embedding database logic directly inside the HTTP layer.

Frontend Database Workspace

The React frontend is primarily an operational interface for the database system.

The workspace provides:

- Table selection
- Table creation
- Table editing
- Table deletion
- Record management
- Schema editing
- Search
- Sorting
- Filtering
- Query execution
- Query Builder
- Query History
- Schema inspection
- Relationship views

The frontend therefore acts as a **database management interface**, rather than being the primary source of business logic.

Schema-Driven User Interface

A particularly strong frontend/database integration is the dynamic relationship between PostgreSQL schema metadata and the user interface.

The `ExpandedTable` implementation interprets column definitions and dynamically determines record-input behaviour.

For example:

INTEGER / DECIMAL

→ numeric input

DATE

→ date input

TIMESTAMP

→ datetime input

BOOLEAN

→ checkbox

VARCHAR / TEXT

→ text input

The interface therefore adapts to the structure of the database rather than requiring a dedicated React form for every custom table.

Data Exploration

The workspace supports database-oriented exploration through:

- Case-insensitive search across row values
- Column-based filtering
- Contains matching
- Equality matching
- Inequality matching
- Prefix matching
- Suffix matching
- Ascending sorting
- Descending sorting
- Null-aware sorting
- Numeric-aware string comparison

This creates a data-management layer above the raw PostgreSQL records.

Database Relationship Management

The application provides a dedicated Relationships module while the backend maintains foreign-key metadata and relationship resolution.

Foreign-key information is derived from PostgreSQL constraints and translated into application-level references.

The system therefore has a database relationship lifecycle spanning:

Schema Definition

→ foreign-key configuration

Validation

→ referenced table/column/type checks

PostgreSQL

→ physical constraint

Introspection

→ relationship discovery

Frontend

→ relationship representation

This is an important example of database metadata being propagated throughout the full application architecture.

Data Integrity and Error Handling

The backend includes PostgreSQL-aware conflict handling.

Database conflict conditions are interpreted using PostgreSQL error codes, including codes associated with:

- Duplicate relations
- Duplicate columns
- Invalid table definitions
- Unique-constraint violations

The system also validates schema definitions before attempting physical database changes.

This creates multiple integrity checkpoints:

Request Validation

→ **Schema Validation**

→ **Database State Validation**

→ **PostgreSQL Constraint Enforcement**

Database Protection Rules

Core system tables such as Products, Categories and Suppliers are treated differently from dynamically managed workspace tables.

The application prevents certain workspace-level operations against protected system resources, such as inappropriate table deletion or renaming.

Additional domain-specific validation exists around relational data operations, including handling categories or suppliers that are associated with products.

The architecture therefore distinguishes between:

System-owned relational resources and **User-defined workspace tables**.

Database Health Monitoring

The API exposes a dedicated health endpoint that performs an actual PostgreSQL connectivity test using:

```
SELECT 1
```

through Prisma.

The endpoint reports:

Connected or **Disconnected** based on the database operation.

This provides a database-aware health check rather than merely confirming that the Express server process is alive.

Database Seeding

The project includes a repeatable Prisma-based database seeding workflow.

The seed process:

- Clears existing Products, Categories and Suppliers.
- Creates eight categories.
- Creates eight suppliers.
- Creates thirty products.
- Generates deterministic SKUs.
- Generates pricing and stock values.
- Establishes category/product relationships.
- Establishes supplier/product relationships.
- Disconnects Prisma cleanly after execution.

The seed data provides an interconnected relational dataset suitable for exercising:

- CRUD operations
 - Relationships
 - Searching
 - Sorting
 - Filtering
 - Query functionality
 - Workspace presentation
 - Database-management workflows
-

Application State and Database Orchestration

At the React application level, `App.jsx` acts as a central orchestration layer.

Initial database resources are loaded concurrently through `Promise.all()` for:

- Products
- Categories
- Suppliers
- Workspace tables

Custom-table records are subsequently loaded using `Promise.allSettled()`, allowing successful table-record requests to remain usable when another request fails.

The application maintains shared database state and provides a central `loadDatabase()` workflow to refresh database-backed views after mutations.

This supports consistency across the different database-management modules.

Session and Operational Controls

The platform implements a client-side inactivity timeout.

Activity events include:

- Mouse movement
- Mouse interaction
- Keyboard interaction
- Scrolling
- Touch interaction

After a defined period of inactivity, the application presents a session-expiry warning with a countdown and subsequently logs the user out.

This is implemented as an application-session control rather than a PostgreSQL authentication mechanism.

Workspace Interaction Engineering

Although secondary to the database architecture, the application provides a spatial workspace for managing database tables.

The implementation incorporates:

- Drag-and-drop functionality
- Sortable interactions
- Draggable/resizable workspace elements
- Table activation
- Row selection
- Double-click interactions
- Expanded table views
- Context-specific edit/delete modes

Libraries including `@dnd-kit` and `react-rnd` support these interactions.

The result is an operational database workspace rather than a conventional collection of CRUD pages.

Frontend Architecture

The frontend is implemented with:

- React 19
- ReactDOM 19
- Vite
- React Router
- Native Fetch
- Lucide icons
- dnd-kit
- react-rnd

The application is decomposed into dedicated modules for:

- Workspace
- Dashboard
- Schema
- Query
- Query History
- Query Builder
- Relationships
- Tables
- Modals
- Search
- Inputs
- Notifications
- Session management

The CSS architecture is modularised into variables, layout, glass effects, animations and global styles, but visual implementation remains secondary to the project's database engineering focus.

API Client Architecture

The frontend uses a central `api()` abstraction built around the native Fetch API.

The abstraction provides:

- Central API base URL configuration
- Environment-based API configuration
- JSON request handling
- Standardised response parsing
- HTTP error detection
- Backend error-message propagation

The API base URL can be configured through:

`VITE_API_URL`

with a local development fallback to the Express API on port 5050.

Dedicated service modules then expose resource-specific operations for Products, Categories, Suppliers, Queries, Schema and Workspace Tables.

Engineering Tooling

The project uses modern JavaScript tooling including:

- Vite 8
- React 19
- React Router 7
- Prisma
- PostgreSQL
- Express
- Oxlint
- Prisma CLI
- ES Modules

The frontend exposes dedicated development, production-build, lint and preview scripts.

The Prisma environment also includes tooling for PostgreSQL setup, Prisma Client/API usage, database configuration, driver adapters and Prisma version management.

QA, Validation and Maintainability

Quality controls are incorporated at multiple architectural levels.

Application Layer

- Controlled React components
- Explicit state transitions
- API error propagation
- Empty states
- Transient notification lifecycle
- Session timeout handling

API Layer

- HTTP status validation
- JSON error handling
- Modular routing
- Environment configuration

Database Layer

- Schema validation
- Type validation
- Identifier validation
- Foreign-key validation
- Constraint validation
- Physical-schema verification
- PostgreSQL error handling
- Transactional schema operations

Development Tooling

- Production build verification
- Oxlint
- Modular service architecture
- Repeatable database seeding

Security-Oriented Database Engineering

The implementation incorporates several database-safety mechanisms.

Dynamic identifiers are explicitly quoted and escaped before being incorporated into generated SQL.

Schema definitions are validated against an explicit allowlist of supported data types.

Foreign-key relationships are validated for referenced-object existence and compatible data types.

Database constraints are represented and enforced at the PostgreSQL layer rather than relying exclusively on frontend behaviour.

Dynamic SQL therefore operates through a controlled schema-definition model rather than unrestricted arbitrary schema fragments.

Architecture Boundaries

The system establishes clear technical boundaries:

React

→ presentation, workspace interaction and application state

API Services

→ HTTP communication

Express

→ routing and backend orchestration

Controllers

→ database/application operations

Prisma

→ structured ORM/database access and workspace metadata

SQL Builder

→ PostgreSQL DDL construction

PostgreSQL

→ authoritative physical relational data structures and integrity constraints

This separation allows the project to combine high-level ORM productivity with low-level database control.

Technical Responsibilities Demonstrated

The project demonstrates breadth across several engineering disciplines while maintaining a strong database focus.

Database Engineer

- PostgreSQL schema design
- Relational modelling
- Runtime DDL

- Constraint management
- Foreign-key architecture
- Schema introspection
- Database metadata
- SQL generation
- Data integrity

Backend Engineer

- Node.js
- Express
- REST API architecture
- Controllers
- Database orchestration
- Error handling
- Transactional operations

SQL Engineer

- PostgreSQL DDL
- `CREATE TABLE`
- `ALTER TABLE`
- Dynamic SQL construction
- Identifier quoting
- Constraint generation
- Type conversion
- Catalog/information-schema queries

Full-Stack Engineer

- React
- API integration
- Shared application state
- Schema-driven interfaces
- Database-backed workflows
- Frontend/backend integration

Product/Workspace Engineering

- Database workflow design
- Query tooling
- Table management
- Data exploration
- Relationship visualisation
- Context-sensitive workspace controls

QA / Maintenance

- Validation
- Error handling
- Database health checks
- Seed-data provisioning
- Build/lint workflows
- Failure-tolerant data loading

Core Skills Demonstrated

Database Engineering

- PostgreSQL
- SQL
- Relational Database Design
- Database Schema Design
- Runtime Schema Evolution
- DDL
- DML
- Schema Introspection
- Database Metadata
- Constraint Engineering
- Referential Integrity
- Foreign-Key Modelling
- Primary-Key Design
- Unique Constraints
- Identity Columns
- Default Values
- Nullability Management

SQL Engineering

- Dynamic SQL Generation
- PostgreSQL DDL
- `CREATE TABLE`
- `ALTER TABLE`
- Constraint SQL
- Identifier Quoting
- SQL Escaping
- Data-Type Conversion
- PostgreSQL Catalog Queries

ORM / Persistence

- Prisma ORM
- Prisma Client
- Prisma PostgreSQL
- Hybrid ORM/Raw SQL Architecture
- Transactional Database Operations
- Database Seeding

Backend

- Node.js
- Express
- REST API Design
- Controller Architecture
- Resource Routing
- Error Handling
- Environment Configuration

Frontend

- React 19
- React Router
- Vite
- Native Fetch
- Schema-Driven UI
- State Management
- Dynamic Forms
- Data Exploration Interfaces

Engineering Practices

- Modular Architecture
- Input Validation
- Runtime Validation
- Database Integrity
- Failure Handling
- Health Monitoring
- Build Verification
- Linting
- Maintainable Service Boundaries

Business and Technical Impact

From a technical perspective, the project demonstrates how a web application can operate as an abstraction layer over a relational database while retaining meaningful control over the underlying PostgreSQL schema.

The platform combines:

Relational Data Management

- **SQL**
- **PostgreSQL**
- **Runtime DDL**
- **Schema Introspection**
- **Constraint Management**
- **ORM Persistence**
- **REST APIs**
- **Schema-Driven Interfaces**

The result is an application capable of bridging the gap between traditional database administration and application-level data management.

From a business systems perspective, the architecture supports scenarios where data structures cannot be assumed to be completely static. The ability to create tables, define relationships, evolve schemas and manage records through a controlled application layer provides a foundation for configurable enterprise data-management workflows.

The strongest technical value lies in the ability to maintain a relationship between **application-level metadata and the physical PostgreSQL database**, while providing controlled mechanisms for schema evolution and data manipulation.
